

Examen de la session ordinaire - Durée (1h30)

Nom et prénom : N° Apogée

Exercice 1 (Entourer la/les bonne(s) réponse(s) : plusieurs réponses correctes sont possibles) (5 pts)

1. Lors de l'utilisation des appels open(), write() et close(), quel problème peut survenir si on oublie close() ?

- A. Le fichier est corrompu B. Fuite de descripteurs de fichiers et dépassement de la limite système
C. L'écriture est annulée D. Le fichier est supprimé automatiquement

2. Qu'affiche ce code ?

```
int pid = fork();  
if (pid > 0) {  
    wait(NULL);  
    printf("Parent\n");  
} else {  
    printf("Child\n");  
}
```

- A. Affiche toujours "Parent" suivi de "Child"
B. Affiche "Child" puis "Parent"
C. Affiche "Parent" puis "Child"
D. Affiche deux fois "Child"

3. Que fait le code suivant ?

```
kill(getpid(), SIGKILL);  
printf("Mort\n");
```

- A. Affiche "Mort" B. Termine immédiatement le processus sans afficher quoi que ce soit
C. Envoie un signal mais continue D. Affiche une erreur

4. Que se passe-t-il si on appelle pthread_exit(NULL) dans le thread principal ?

- A. Le thread principal termine mais le processus reste vivant si d'autres threads existent
B. Le processus entier se termine
C. Rien, c'est ignoré D. Provoque une segmentation fault

5. On souhaite créer 5 threads avec pthread_t threads[5];. Quelle est la meilleure façon d'attendre que tous les threads terminent correctement ?

- A. Appeler pthread_exit(NULL); à la fin du main() pour que tous les threads terminent.
B. Utiliser une boucle for avec pthread_detach(threads[i]); pour s'assurer qu'ils se terminent automatiquement.
C. Utiliser une boucle for avec pthread_join(threads[i], NULL); pour bloquer jusqu'à la fin de chaque thread.
D. Utiliser wait(NULL); dans une boucle, une fois les threads lancés.

Exercice 2 (Questions de compréhension : donnez une réponse brève et bien précise) (3 pts)

1. Quelle est la différence entre kill(pid, sig) et raise(sig) ?

.....
.....

2. Peut-on exécuter un programme dans un processus existant sans créer un nouveau processus ? Si oui, comment ?

.....
.....

3. Que se passe-t-il si un thread ne libère pas un mutex ?

.....
.....

Exercice 3 : (5 pts)

```
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
int main() {
    int fd = open("test.txt", O_CREAT | O_WRONLY , 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }
    pid_t pid = fork();
    if (pid == 0) {
        write(fd, "Fils\n", 5);
    } else {
        write(fd, "Père\n", 5);
    }
    close(fd);
    return 0;
}
```

1. Quel processus sera exécuté en premier ?
2. Que contiendra le fichier test.txt après l'exécution ? Pourquoi ?
3. Modifier le programme pour obliger le père à s'exécuter en premier.

Exercice 4 : (6 pts)

Ecrire un programme en C qui simule un système qui surveille la température d'une machine et active un refroidissement si nécessaire.

- Un **thread capteur** mesure la température toutes les 3 secondes (simule une valeur aléatoire entre 15°C et 35°C).
- Un **thread contrôleur** vérifie la température et allume le refroidissement si elle dépasse 30°C, sinon il l'éteint.
- Toutes les 10 secondes, un **signal** est envoyé pour afficher la température actuelle et l'état du refroidissement.
- Protégez l'accès aux données partagées avec un **mutex** pour éviter les conflits.
- Le programme doit tourner pendant 30 secondes.

N.B : 1 point pour une bonne présentation / écriture